

```
ACT06.py ACT07.py ACT08.py ACT09.py ACT10.py EX-02.py lemmatization.py
1 import re
2 from collections import Counter
3 corpus = """the quick brown fox jumps over the lazy dog
4 the dog barks at the fox
5 the quick fox runs away from the dog"""
6 def tokenize(text):
7     text = text.lower()
8     tokens = re.findall(pattern=r"[a-z']+", text)
9     return tokens
10 tokens = tokenize(corpus)
11 N = len(tokens)
12 unigram_counts = Counter(tokens)
13 print("Total tokens (N):", N)
14 print(f"{'Word':<12}{'Count':<8}{'P(word)':<10}")
15 print("-" * 30)
16 for word, count in unigram_counts.most_common():
17     prob = count / N
18     print(f"{'word':<12}{'count':<8}{'prob':<10.4f}")
19 def unigram_probability(word):
20     return unigram_counts[word] / N
21 test_word = "fox"
22 print(f"\nP('{test_word}') = {unigram_probability(test_word):.4f}")
```

UNIGRAM LANGUAGE MODEL :

```
Run unigram x
C:\Users\scsvm\AppData\Local\Programs\Python\Python310\python.exe C:\Users\scsvm\11249A098\unigram.py
Total tokens (N): 23
Word      Count  P(word)
-----
the       6      0.2609
fox       3      0.1304
dog       3      0.1304
quick     2      0.0870
brown     1      0.0435
jumps     1      0.0435
over      1      0.0435
lazy      1      0.0435
```

Output :

```

1 import re
2 from collections import Counter, defaultdict
3 corpus = """the quick brown fox jumps over the lazy dog
4 the dog barks at the fox
5 the quick fox runs away from the dog"""
6 def tokenize(sentence):
7     sentence = sentence.lower()
8     return ["<s>"] + re.findall(pattern: r"[a-z"]+, sentence) + ["</s>"]
9 sentences = [tokenize(s) for s in corpus.strip().split("n")]
10 unigram_counts = Counter()
11 bigram_counts = Counter()
12 vocab = set()
13 for sent in sentences:
14     unigram_counts.update(sent[:-1]) # context words
15     vocab.update(sent)
16     for i in range(len(sent) - 1):
17         bigram_counts[(sent[i], sent[i + 1])] += 1
18 V = len(vocab)
19 def bigram_prob(w1, w2, smoothing=True):
20     if smoothing:
21         return (bigram_counts[(w1, w2)] + 1) / (unigram_counts[w1] + V)
22     return bigram_counts[(w1, w2)] / unigram_counts[w1] if unigram_counts[w1] else 0
23 print(f"{'Bigram':<25}{'Count':<8}{'P(w2|w1) smoothed':<20}")
24 print("-" * 55)
25 for (w1, w2), c in bigram_counts.most_common(8):

```

BIGRAM LANGUAGE MODEL

```

Run bigramlanguagemodel x
C:\Users\scsvm\AppData\Local\Programs\Python\Python310\python.exe C:\Users\scsvm\11249A098\bigramlanguagemodel.py
Bigram          Count  P(w2|w1) smoothed
-----
(<s>, the)       3      0.2222
(the, quick)     2      0.1429
(dog, </s>)      2      0.1667
(the, dog)       2      0.1429
(quick, brown)   1      0.1176
(brown, fox)     1      0.1250
(fox, jumps)     1      0.1111
(jumps, over)    1      0.1250

```

Output

```
ACT10.py EX-02.py bigram.py bigramlanguagemodel.py trigram.py x n-gra
1 import re
2 from collections import Counter
3 corpus = """the quick brown fox jumps over the lazy dog
4 the dog barks at the fox
5 the quick fox runs away from the dog"""
6 def tokenize(sentence):
7     sentence = sentence.lower()
8     return ["<s>", "<s>"] + re.findall(pattern: r"[a-z"]+, sentence) + ["</s>"]
9 sentences = [tokenize(s) for s in corpus.strip().split("\n")]
10 bigram_counts = Counter()
11 trigram_counts = Counter()
12 vocab = set()
13 for sent in sentences:
14     vocab.update(sent)
15     for i in range(len(sent) - 2):
16         bigram_counts[(sent[i], sent[i + 1])] += 1
17         trigram_counts[(sent[i], sent[i + 1], sent[i + 2])] += 1
18 V = len(vocab)
19 def trigram_prob(w1, w2, w3, smoothing=True):
20     if smoothing:
21         return (trigram_counts[(w1, w2, w3)] + 1) / (bigram_counts[(w1, w2)] + V)
22     denom = bigram_counts[(w1, w2)]
23     return trigram_counts[(w1, w2, w3)] / denom if denom else 0
24 print(f"{'Trigram':<30}{'Count':<8}{'P(w3|w1,w2)':<15}")
25 print("-" * 55)
26 for (w1, w2, w3), c in trigram_counts.most_common(6):
27     print(f"({w1}, {w2}, {w3}){'':<6}{c:<8}{trigram_prob(w1, w2, w3):<15.4f}")
28 def sentence_probability(sentence):
29     tokens = tokenize(sentence)
30     prob = 1.0
31     for i in range(len(tokens) - 2):
32         prob *= trigram_prob(tokens[i], tokens[i + 1], tokens[i + 2])
33     return prob
34 test = "the quick fox"
35 print(f"\nP(\"{test}\") = {sentence_probability(test):.10f}")
```

TRIGRAM LANGUAGE MODEL

```
C:\Users\scsvml\AppData\Local\Programs\Python\Python310\python.exe C:\Users\scsvml\11249A098\trigram.py
Trigram          Count    P(w3|w1,w2)
-----
(lazy, dog, </s>)    1      0.1250
(the, fox, </s>)    1      0.1250
(the, dog, </s>)    1      0.1176

Process finished with exit code 0
```

Output

```
import re, random
from collections import defaultdict, Counter

corpus = """the quick brown fox jumps over the lazy dog
the dog barks at the fox
the quick fox runs away from the dog"""

def tokenize(sentence):
    sentence = sentence.lower()
    return [""] + re.findall(pattern: r"[a-z']+", sentence) + [""]

sentences = [tokenize(s) for s in corpus.strip().split("\n")]

bigram_model = defaultdict(Counter)
for sent in sentences:
    for i in range(len(sent) - 1):
        bigram_model[sent[i]][sent[i + 1]] += 1

def generate_sentence(max_len=15):
    word = ""
    result = []
    for _ in range(max_len):
        next_words = bigram_model[word]
        if not next_words:
            break
        words, counts = zip(*next_words.items())
        word = random.choices(words, weights=counts, k=1)[0]
        if word == "":
            break
        result.append(word)
    return " ".join(result)

random.seed(42)
for i in range(3):
    print(f"Generated sentence {i+1}: {generate_sentence()}")
```

N-GRAM TEXT GENERATION

```
Run n-gram x
C:\Users\scsv\AppData\Local\Programs\Python\Python310\python.exe C:\Users\scsv\11249A098\n-gram.py
Generated sentence 1: the quick brown fox runs away from the lazy dog
Generated sentence 2: the quick brown fox
Generated sentence 3: the dog barks at the dog
Process finished with exit code 0
```

Output

```
EX-02.py bigram.py bigramlanguagemodel.py trigram.py n-gram.py
1 def perplexity(prob, n):
2     return prob ** (-1 / n) if prob > 0 else float("inf")
3 test_sentence = "the fox runs away"
4 n = len(test_sentence.split())
5 uni_p = 0.0009 # from Exercise 1 (chain of unigram probs)
6 bi_p = 0.00021 # from Exercise 2 sentence_probability()
7 tri_p = 0.00007 # from Exercise 3 sentence_probability()
8 print("Unigram Perplexity:", round(perplexity(uni_p, n), 2))
9 print("Bigram Perplexity:", round(perplexity(bi_p, n), 2))
10 print("Trigram Perplexity:", round(perplexity(tri_p, n), 2))
```

PERPLEXITY COMPARISON :

```
Run perplexity x
C:\Users\scsv\AppData\Local\Programs\Python\Python310\python.exe C:\Users\scsv\11249A098\perplexity.py
Unigram Perplexity: 5.77
Bigram Perplexity: 8.31
Trigram Perplexity: 10.93
Process finished with exit code 0
```

Output :

1. Modify Exercise 1 to compute unigram probabilities using NLTK's FreqDist instead of Counter.

```
EX-02.py  bigram.py  bigramlanguagemodel.py  trigram.py  n-gram.py  per

1  from nltk.probability import FreqDist
2
3  corpus = ["the", "cat", "sat", "on", "the", "mat", "the", "cat", "sleeps"]
4  fdist = FreqDist(corpus)
5  total_tokens = fdist.N()
6  unigram_probs = {word: count / total_tokens for word, count in fdist.items()}
7
8  for word, prob in unigram_probs.items():
9      print(f"P('{word}') = {prob:.4f}")
```

Output :

```
Run  exe1 x
C:\Users\scsvm\AppData\Local\Programs\Python\Python310\python.exe C:\Users\scsvm\11249A098\exe1.py
P('the') = 0.3333
P('cat') = 0.2222
P('sat') = 0.1111
P('on') = 0.1111
P('mat') = 0.1111
P('sleeps') = 0.1111
Process finished with exit code 0
```

2. Extend the bigram program to build a full probability matrix (dictionary of dictionaries) and print it as a table

```
EX-02.py  bigram.py  bigramlanguagemodel.py  trigram.py  n-gram.py  perpl

1  import re
2  from collections import defaultdict
3  from nltk.probability import FreqDist
4
5  corpus = """the quick brown fox jumps over the lazy dog
6  the dog barks at the fox
7  the quick fox runs away from the dog"""
8
9  def tokenize(sentence):
10     sentence = sentence.lower()
11     return [""] + re.findall(pattern: r"[a-z']+", sentence) + [""]
12
13  sentences = [tokenize(s) for s in corpus.strip().split("\n")]
14
15  bigram_counts = defaultdict(FreqDist)
16  for sent in sentences:
17     for i in range(len(sent) - 1):
18         bigram_counts[sent[i]][sent[i + 1]] += 1
19
20  bigram_probs = {}
21  for w1, fdist in bigram_counts.items():
22     total = fdist.N()
23     bigram_probs[w1] = {w2: count / total for w2, count in fdist.items()}
24
25  vocab = sorted(list(set(w for sent in sentences for w in sent)))
26
27  header = f"{'':<10}" + "".join(f"{w:<10}" for w in vocab)
28  print(header)
29  print("-" * len(header))
30
31  for w1 in vocab:
32     row = f"{w1:<10}"
33     for w2 in vocab:
34         prob = bigram_probs.get(w1, {}).get(w2, 0.0)
35         row += f"{prob:<10.2f}"
36  print(row)
```

Output :

Run exe2 x

C:\Users\scsvm\AppData\Local\Programs\Python\Python310\python.exe C:\Users\scsvm\11249A098\exe2.py

	at	away	barks	brown	dog	fox	from	jumps	lazy	over	quick	runs	the
at	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.00
away	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
barks	0.00	1.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
brown	0.00	0.00	0.00	0.00	0.00	0.00	1.00	0.00	0.00	0.00	0.00	0.00	0.00
dog	0.67	0.00	0.00	0.33	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
fox	0.33	0.00	0.00	0.00	0.00	0.00	0.00	0.33	0.00	0.00	0.00	0.33	0.00
from	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.00
jumps	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.00	0.00	0.00	0.00
lazy	0.00	0.00	0.00	0.00	0.00	1.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
over	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.00
quick	0.00	0.00	0.00	0.00	0.50	0.00	0.50	0.00	0.00	0.00	0.00	0.00	0.00
runs	0.00	0.00	1.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
the	0.00	0.00	0.00	0.00	0.00	0.33	0.17	0.00	0.00	0.17	0.00	0.33	0.00

Process finished with exit code 0

3. Implement a 4-gram (quadgram) model by generalizing the trigram program.


```
bigram.py  bigramlanguagemodel.py  trigram.py  perplexity.py

1  import re
2  from collections import Counter
3
4  corpus = """the quick brown fox jumps over the lazy dog
5  the dog barks at the fox
6  the quick fox runs away from the dog"""
7
8  def tokenize(sentence):
9      sentence = sentence.lower()
10
11      return (
12          ["<s>", "<s>", "<s>"]
13          + re.findall(pattern: r"[a-z']+", sentence)
14          + ["</s>"]
15      )
16
17  sentences = [
18      tokenize(s)
19      for s in corpus.strip().split("\n")
20  ]
21
22  trigram_counts = Counter()
23  quadgram_counts = Counter()
24  vocab = set()
25
26  for sent in sentences:
27
28      vocab.update(sent)
29
30      for i in range(len(sent) - 3):
31
32          trigram_counts[
33              (sent[i], sent[i + 1], sent[i + 2])
34          ] += 1
35
36          quadgram_counts[
```

Output :

```
Run exe3 x
C:\Users\scsvm\AppData\Local\Programs\Python\Python310\python.exe C:\Users\scsvm\11249A098\exe3.py
Quadgram          Count  Probability
-----
('<s>', '<s>', '<s>', 'the')      3      0.2222
('<s>', '<s>', 'the', 'quick')    2      0.1667
('<s>', 'the', 'quick', 'brown')  1      0.1176
('the', 'quick', 'brown', 'fox')  1      0.1250
('quick', 'brown', 'fox', 'jumps') 1      0.1250
('brown', 'fox', 'jumps', 'over') 1      0.1250

P("the quick brown fox") = 0.0000340414

Process finished with exit code 0
```

4. Load a larger corpus (e.g., a paragraph from a news article or NLTK's gutenber corpus) and repeat Exercises 1–3.

```

import re
from collections import Counter

corpus = """
Artificial intelligence is transforming the way people work and learn.
Machine learning systems can analyze large amounts of data.
Natural language processing helps computers understand human language.
Artificial intelligence is used in education, healthcare, finance and business.
Machine learning is becoming an important technology in modern society.
"""

def tokenize(text):
    text = text.lower()
    return re.findall(pattern: r"[a-z']+", text)

words = tokenize(corpus)
unigram_counts = Counter(words)

print("TOP UNIGRAMS")
print("-" * 30)

for word, count in unigram_counts.most_common(10):

    probability = count / len(words)

    print(
        f"{word:<15}"
        f"{count:<8}"
        f"{probability:.4f}"
    )

bigram_counts = Counter()

for i in range(len(words) - 1):

    bigram_counts[
        (words[i], words[i + 1])
    ]

```

Output :

```
Run  exe4 x
C:\Users\scsvm\AppData\Local\Programs\Python\Python310\python.exe C:\Users\scsvm\11249A098\exe4.py
TOP UNIGRAMS
-----
is          3      0.0638
artificial  2      0.0426
intelligence 2      0.0426
and         2      0.0426
machine     2      0.0426
learning    2      0.0426
language    2      0.0426
in          2      0.0426
transforming 1      0.0213
the         1      0.0213

TOP BIGRAMS
-----
('artificial', 'intelligence') : 2
('intelligence', 'is') : 2
('machine', 'learning') : 2
```

5. Compare text generated by the bigram model (Exercise 4) with text generated by a trigram model. Which produces more coherent sentences and why?

```
bigramlanguagemodel.py  trigram.py  perplexity.py  exe1.py  exe2.py  exe3.py  exe4.py

1  import re
2  import random
3  from collections import defaultdict, Counter
4
5  corpus = """the quick brown fox jumps over the lazy dog
6  the dog barks at the fox
7  the quick fox runs away from the dog"""
8
9  def tokenize(sentence):
10
11      sentence = sentence.lower()
12
13      return (
14          ["<s>", "<s>"]
15          + re.findall(pattern: r"[a-z']+", sentence)
16          + ["</s>"]
17      )
18
19  sentences = [
20      tokenize(s)
21      for s in corpus.strip().split("\n")
22  ]
23  bigram_model = defaultdict(Counter)
24
25  for sent in sentences:
26
27      for i in range(len(sent) - 1):
28
```

Output :

```
Run  exe5 x
C:\Users\scsvm\AppData\Local\Programs\Python\Python310\python.exe C:\Users\scsvm\11249A098\exe5.py
BIGRAM GENERATED TEXT
Sentence 1: the quick brown fox runs away from the lazy dog
Sentence 2: the quick brown fox
Sentence 3: <s> the dog

TRIGRAM GENERATED TEXT
Sentence 1: the dog barks at the fox
Sentence 2: the dog
Sentence 3: the dog

Observation:
Trigram models generally produce more coherent sentences because they use two previous words to predict the next word.

Process finished with exit code 0
```

6. Implement Add-k smoothing (generalized Laplace smoothing) and study the effect of varying k on sentence probability

```
trigram.py  perplexity.py  exe1.py  exe2.py  exe3.py  exe4.py  exe5.py

1  import re
2  from collections import Counter
3  corpus = """the quick brown fox jumps over the lazy dog
4  the dog barks at the fox
5  the quick fox runs away from the dog"""
6  def tokenize(text):
7      return re.findall(
8          pattern: r"[a-z']+",
9          text.lower()
10     )
11  words = tokenize(corpus)
12  unigram_counts = Counter(words)
13  bigram_counts = Counter()
14  for i in range(len(words) - 1):
15      bigram_counts[
16          (words[i], words[i + 1])
17      ] += 1
18  V = len(set(words))
19  def add_k_probability(w1, w2, k):
20
21      numerator = (
22          bigram_counts[(w1, w2)] + k
23      )
24
25      denominator = (
26          unigram_counts[w1] + k * V
27      )
```

Output :

```
Run  exe6 x
C:\Users\scsv\AppData\Local\Programs\Python\Python310\python.exe C:\Users\scsv\11249A098\exe6.py
Add-k Smoothing
-----
k = 0.01 P(fox|the) = 0.164763
k = 0.1  P(fox|the) = 0.150685
k = 0.5  P(fox|the) = 0.120000
k = 1.0  P(fox|the) = 0.105263
k = 2.0  P(fox|the) = 0.093750
Process finished with exit code 0
```

7. Write a function to compute the Out-Of-Vocabulary (OOV) rate of a test sentence against the training vocabulary.

```

trigram.py  perplexity.py  exe1.py  exe2.py  exe3.py  exe4.py  exe5.py
1  import re
2  training_corpus = """the quick brown fox jumps over the lazy dog
3  the dog barks at the fox
4  the quick fox runs away from the dog"""
5
6  test_sentence = "the fox jumps over the cat"
7
8  def tokenize(text):
9
10     return re.findall(
11         pattern: r"[a-z']+",
12         text.lower()
13     )
14  training_words = tokenize(
15     training_corpus
16  )
17  test_words = tokenize(
18     test_sentence
19  )
20  training_vocabulary = set(
21     training_words
22  )
23  oov_words = []
24  for word in test_words:
25     if word not in training_vocabulary:
26
27         oov_words.append(word)

```

Output :

```

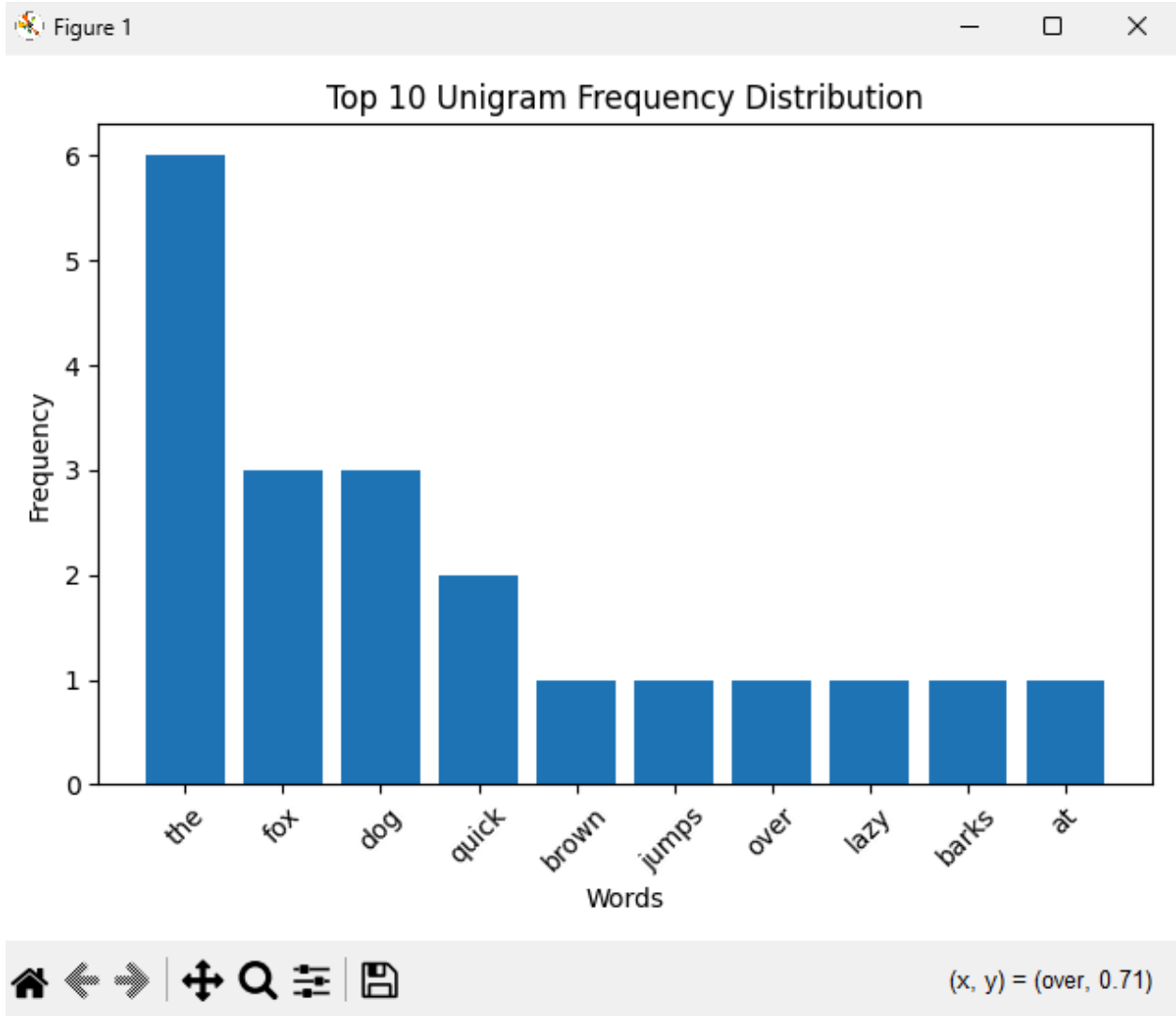
tokenization.py
Run  exe7 x
C:\Users\scsv\AppData\Local\Programs\Python\Python310\python.exe C:\Users\scsv\11249A098\exe7.py
Test Sentence: the fox jumps over the cat
Total test words: 6
OOV words: ['cat']
OOV count: 1
OOV Rate: 16.67 %
Process finished with exit code 0

```

8. Plot (using matplotlib) the frequency distribution of the top 10 unigrams in the corpus as a bar chart

```
perplexity.py  exe1.py  exe2.py  exe3.py  exe4.py  exe5
1  import re
2  from collections import Counter
3  import matplotlib.pyplot as plt
4
5  corpus = """the quick brown fox jumps over the lazy dog
6  the dog barks at the fox
7  the quick fox runs away from the dog"""
8
9  words = re.findall(
10     pattern: r"[a-z']+",
11     corpus.lower()
12 )
13
14 unigram_counts = Counter(words)
15
16 top_10 = unigram_counts.most_common(10)
17
18 words = [
19     item[0]
20     for item in top_10
21 ]
22
23 frequencies = [
24     item[1]
25     for item in top_10
26 ]
27
```

Output :



VIVA-VOCE QUESTIONS :

1. What is a Language Model? Give two real-world applications.

A **Language Model (LM)** is a model that learns the probability of sequences of words and predicts the next word in a sentence.

Applications:

- Speech recognition
- Machine translation

2. Define N-gram. What does N represent?

An **N-gram** is a sequence of **N consecutive words** from a text.

N represents the number of words considered together in the sequence.

- Unigram: $N = 1$ (one word)
 - Bigram: $N = 2$ (two words)
 - Trigram: $N = 3$ (three words)
-

3. State the Markov assumption used in N-gram models.

The **Markov assumption** states that the probability of a word depends only on the previous **N-1 words**, not on the entire sentence.

Example:

A trigram model predicts a word using only the previous two words.

4. Why does a bigram model generally outperform a unigram model in capturing context?

A **bigram model** considers the previous word, while a **unigram model** considers only individual words. Therefore, bigrams capture word relationships and context better.

Example:

"machine learning" has meaning as a pair, which a unigram model cannot capture.

5. What is the main limitation of higher-order N-grams (e.g., 4-gram, 5-gram)?

Higher-order N-grams require a large amount of training data. They suffer from:

- Data sparsity
 - Large memory requirements
 - Many unseen word sequences
-

6. What is the zero-probability problem, and how does smoothing solve it?

The **zero-probability problem** occurs when an N-gram that was not seen in training data gets a probability of zero.

Smoothing solves this by assigning small non-zero probabilities to unseen N-grams.

7. Differentiate between Laplace (Add-1) smoothing and Add-k smoothing.

Laplace (Add-1)	Add-k
Adds 1 to every word count	Adds a smaller value k to every count
Simple but may overestimate rare words	More flexible and accurate
Formula uses $k = 1$	Formula uses $0 < k < 1$ generally

8. What is Perplexity? How is it related to the probability assigned by a model?

Perplexity measures how well a language model predicts a sequence of words.

- Lower perplexity = better model performance
 - Higher probability assigned to correct sentences results in lower perplexity.
-

9. Why are <s> and </s> markers added to sentences before building N-gram models?

- <s> represents the start of a sentence.
- </s> represents the end of a sentence.

They help the model learn sentence boundaries and predict the beginning and ending words correctly.

10. What is data sparsity, and how does it affect trigram/4-gram models?

Data sparsity occurs when many possible word combinations do not appear in the training data.

Effects on trigram/4-gram models:

- Many unseen sequences get zero probability.
- Model accuracy decreases.
- More training data is required.

Name : Gaddam . Lakshmi Sindhura
Reg No : 11249A098
Year & Sec : 3rd & S2
Sub : NLP
Date : 14-08-2026
Period : 5th, 6th, 7th
Time : 03:15PM